

```
}  
}  
  
Enter a byte value: 100  
Enter a short value: 20000  
Enter an int value: 500000  
Enter a long value: 9876543210  
Enter a float value: 12.5  
Enter a double value: 45.6789  
  
----- Values Entered -----  
Byte : 100  
Short : 20000  
Int : 500000  
Long : 9876543210  
Float : 12.5  
Double : 45.6789
```

Non-Primitive (Reference) Data Types

Non-primitive data types are derived from primitive types or created by programmers.

One-Dimensional Arrays

```
public class Main {  
    public static void main(String[] args) {  
  
        int[] arr = {10, 20, 30, 40, 50};  
  
        System.out.println("Array Elements:");  
  
        for (int i = 0; i < arr.length; i++) {  
            System.out.println(arr[i]);  
        }  
    }  
}
```

```

class Testarray{
public static void main(String args[]){
    int a[]=new int[5];//declaration and instantiation

    a[0]=10;//initialization
    a[1]=20;
    a[2]=70;
    a[3]=40;
    a[4]=50;

    //traversing array
    for(int i=0;i<a.length;i++)//length is the of array
        System.out.println(a[i]);
}
}

```

Declaration	Valid?
int[] a;	✓ Recommended
int a[];	✓
int[] a, b;	✓ Both arrays
int a[], b[];	✓ Both arrays
int a[], b;	✓ a array, b int
int a[10];	❑ Invalid in Java

Multidimensional Arrays

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
    }
}

```

```

int[][] a = new int[3][3];

// Input
System.out.println("Enter 9 elements:");

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        a[i][j] = sc.nextInt();
    }
}

// Display
System.out.println("\nMatrix:");

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        System.out.print(a[i][j] + " ");
    }
    System.out.println();
}

sc.close();
}

```

```

Input
10 20 30
40 50 60
70 80 90
Output
Matrix:
10 20 30
40 50 60
70 80 90

```

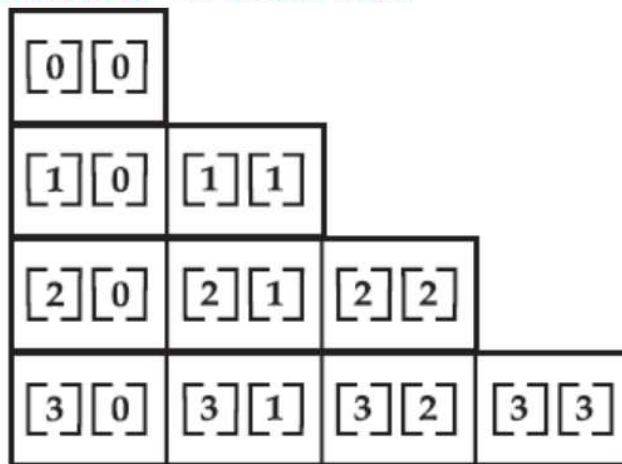
Declaration	Valid?
int[][] a;	✓ Recommended

<code>int a[];</code>	✓
<code>int[] a[];</code>	✓
<code>int[][] a, b;</code>	✓ Both 2D arrays
<code>int[] a[], b;</code>	✓ a 2D array, b 1D array
<code>int[][] a = new int[3][4];</code>	✓
<code>int a[][] = new int[3][4];</code>	✓

Jagged Array in Java

A Jagged Array (also called a **Ragged Array**) is a 2D array in which each row can have a different number of columns.

- Creating odd number of columns in a 2D array, it is known as a jagged array.
- In other words, it is an array of arrays with **different number of columns**.



```
public class Main {
    public static void main(String[] args) {

        int[][] a = {
            {10, 20, 30},
            {40, 50, 60, 70, 80},
            {90, 100}
        }
    }
}
```

```

};

for (int i = 0; i < a.length; i++) {
    for (int j = 0; j < a[i].length; j++) {
        System.out.print(a[i][j] + " ");
    }
    System.out.println();
}
}
}
}

```

```

10 20 30
40 50 60 70 80
90 100

```

Creating Jagged Array Using new

```

public class Main {
    public static void main(String[] args) {

        int[][] a = new int[3][];

        a[0] = new int[2];
        a[1] = new int[4];
        a[2] = new int[3];

        a[0][0] = 10;
        a[0][1] = 20;

        a[1][0] = 30;
        a[1][1] = 40;
        a[1][2] = 50;
        a[1][3] = 60;

        a[2][0] = 70;
        a[2][1] = 80;
        a[2][2] = 90;

        for (int i = 0; i < a.length; i++) {
            for (int j = 0; j < a[i].length; j++) {
                System.out.print(a[i][j] + " ");
            }
            System.out.println();
        }
    }
}

```

```

    }
}

10 20
30 40 50 60
70 80 90

```

3D Array

```

// Demonstrate a three-dimensional array.
class ThreeDMatrix {
public static void main(String args[]) {
    int threeD[][][] = new int[3][4][5];
    int i, j, k;
    for(i=0; i<3; i++)
        for(j=0; j<4; j++)
            for(k=0; k<5; k++)
                threeD[i][j][k] = i * j * k;

    for(i=0; i<3; i++) {
        for(j=0; j<4; j++) {
            for(k=0; k<5; k++)
                System.out.print(threeD[i][j][k] + " ");
            System.out.println();
        }
        System.out.println();
    }
}
}

```

```

output:
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0

0 0 0 0 0
0 1 2 3 4
0 2 4 6 8
0 3 6 9 12

0 0 0 0 0
0 2 4 6 8
0 4 8 12 16
0 6 12 18 24

```

OPERATORS

Operators: Arithmetic Operators, The Bitwise Operators, Relational Operators, Boolean Logic operators, The assignment operator, The ? Operator, Operator Precedence, Using Parentheses.

Arithmetic operators